

~\Dropbox\Active\20220331_PhantomSuzuki\Guide_n_Record_v2_2\Guide_n_ReadMotion.cpp

```

1 // Guide and Record Version 2.2
2 // 2023.07.10
3 // by M.Teranishi
4 // Change log :
5 // flexible recording time limit.
6 /*****
7
8 Copyright (c) 2004 SensAble Technologies, Inc. All rights reserved.
9
10 OpenHaptics(TM) toolkit. The material embodied in this software and use of
11 this software is subject to the terms and conditions of the clickthrough
12 Development License Agreement.
13
14 For questions, comments or bug reports, go to forums at:
15     http://dsc.sensable.com
16
17 Module Name:
18
19     CommandJointTorque.cpp
20
21 Description:
22
23     This example demonstrates commanding joint torques to the Phantom device.
24 *****/
25 #ifdef _WIN64
26 #pragma warning (disable:4996)
27 #endif
28
29 #include <stdio.h>
30 #include <stdlib.h>
31 #include <assert.h>
32
33 #if defined(WIN32)
34 # include <windows.h>
35 # include <conio.h>
36 #pragma comment(lib, "winmm.lib")
37 #else
38 #include <time.h>
39 # include "conio.h"
40 # include <string.h>
41 #define FALSE 0
42 #define TRUE 1
43 #endif
44 #include <direct.h>
45
46 #include <HD/hd.h>
47 #include <HDU/hduError.h>
48 #include <HDU/hduVector.h>
49
50 static bool ReplayMode = false;
51 static bool ReplayReady = false; // whether replay data loaded or not.

```

```

52  /* # of record */
53  #define LEN_REC 10000
54  /* interval for recording [ms]*/
55  #define INTV_REC 10
56  // filename for record
57  #define FNAME_REC "record.csv" // up dir due to debug exec.
58  // filename for playback
59  #define FNAME_REPLAY "replay.csv"
60
61  #define FNAME_REPLAY_RESAMPLE "replay_resampled.csv"
62
63  // line buffer length for csv reading.
64  #define LEN_LINE_CSV 1024
65  // Maximum length of replay data (raw)
66  #define MAX_LEN_REPLAY_RAW 20000
67  // re-sampling interval[ms] for replay data
68  #define INTV_RESAMPLE_REPLAY_MS 10
69  // Maximum length of replay data (resample)
70  #define MAX_LEN_REPLAY_RESAMPLE 10000
71
72  #define BUFSIZE MAX_PATH
73
74
75  static int end_idx_resample;
76  static int end_idx_record;
77
78
79  static hduVector3Dd rec_position[LEN_REC];
80  static hduVector3Dd rec_gimbalAngles[LEN_REC];
81  static DWORD rec_timeidx[LEN_REC];
82  static DWORD time_start;
83
84  static hduVector3Dd replay_position_raw[MAX_LEN_REPLAY_RAW];
85  static DWORD replay_timeidx_raw[MAX_LEN_REPLAY_RAW];
86
87  static hduVector3Dd replay_position_resample[MAX_LEN_REPLAY_RESAMPLE];
88  static DWORD replay_timeidx_resample[MAX_LEN_REPLAY_RESAMPLE];
89
90  static hduVector3Dd wellPos;
91
92  HDCallbackCode HDCALLBACK guideAndRecordCallback(void *data);
93  HDSchedulerHandle hGravityWell = HD_INVALID_HANDLE;
94
95  void mainLoop(void);
96  bool initDemo(void);
97
98  void PrintCwd()
99  {
100     char* buffer;
101     // Get the current working directory:
102     if ((buffer = _getcwd(NULL, 0)) == NULL)
103         perror("_getcwd error");
104     else
105         {

```

```

106     printf("CWD : %s \nLength: %zu\n", buffer, strlen(buffer));
107     free(buffer);
108 }
109
110 return;
111
112 }
113
114 void PrintHelp()
115 {
116     static const char help[] = {\
117 "GuideAndRecord Help\n\
118 ---\n\
119 L: Load replay data\n\
120 G: Guide mode start\n\
121 R: Record mode start\n\
122 D: Load Recorded data\n\
123 P: Prints device state\n\
124 C: Continuously prints device state\n\
125 A: Load and resample RAW replay data \n\
126 W: Write resampled replay data(debug)\n\
127 H: Prints help menu\n\
128 Q: Quits the program\n\
129 ---"};
130
131     printf("\n%s\n", help);
132     PrintCwd();
133 }
134
135 /* Synchronization structure. */
136 typedef struct
137 {
138     HDdouble position[3]; /* mm */
139     HDdouble gimbalAngles[3]; /* rad */
140 } DeviceStateStruct;
141
142 /*****
143  Callback that retrieves state.
144  *****/
145 HDCallbackCode HDCALLBACK GetDeviceStateCallback(void *pUserData)
146 {
147     DeviceStateStruct *pState = (DeviceStateStruct *) pUserData;
148
149     hdGetDoublev(HD_CURRENT_POSITION, pState->position);
150     hdGetDoublev(HD_CURRENT_GIMBAL_ANGLES, pState->gimbalAngles);
151
152     return HD_CALLBACK_DONE;
153 }
154
155 /*****
156  Callback that retrieves state.
157  *****/
158 void PrintDeviceState(HDboolean bContinuous)
159 {

```

```

160     int i;
161     DeviceStateStruct state;
162
163     memset(&state, 0, sizeof(DeviceStateStruct));
164
165     do
166     {
167         hdScheduleSynchronous(GetDeviceStateCallback, &state,
168             HD_DEFAULT_SCHEDULER_PRIORITY);
169
170         printf("\n");
171
172         printf("Current position (mm):");
173         for (i = 0; i < 3; i++)
174         {
175             printf(" %f", state.position[i]);
176         }
177         printf("\n");
178
179         printf("Current Gimbal Angles (rad):");
180         for (i = 0; i < 3; i++)
181         {
182             printf(" %f", state.gimbalAngles[i]);
183         }
184         printf("\n");
185
186         if (bContinuous)
187         {
188             #if defined(WIN32)
189                 Sleep(500);
190             #elif defined(linux)
191                 struct timespec timeOut;
192                 timeOut.tv_sec = 0;
193                 timeOut.tv_nsec = 5*100000000;
194                 nanosleep(&timeOut, NULL);
195             #endif
196         }
197     } while (!_kbhit() && bContinuous);
198 }
199
200
201
202 /*****
203  Callback that record state.
204  *****/
205 void RecordDeviceState()
206 {
207     /* # of record */
208     // #define LEN_REC 10000
209     /* interval for recording [ms]*/
210     // #define INTV_REC 10
211     float lim_rec_time = LEN_REC * INTV_REC * 0.001;
212     int i;
213     int t;

```

```

214     double end_time;
215     DWORD now, time_next;
216
217     FILE* fp_rec_csv = NULL;
218
219     DeviceStateStruct state;
220
221     memset(&state, 0, sizeof(DeviceStateStruct));
222     //end_idx_record = end_idx_resample;
223     //end_time = end_idx_record * INTV_REC / 1000.0; //sec
224
225     fprintf(stderr, "Now I record your motion in %f seconds.\n", lim_rec_time);
226
227
228     fprintf(stderr, "Recording mode starts after 3 sec.count.");
229     for (i = 3; i > 0; --i) {
230         fprintf(stderr, "%d  ", i);
231         Sleep(1000);
232     }
233     fprintf(stderr, "...go!\n");
234     fprintf(stderr, "### hit any key to end record ###\n");
235
236     now = time_start = timeGetTime();
237
238     hdScheduleSynchronous(GetDeviceStateCallback, &state,
239         HD_DEFAULT_SCHEDULER_PRIORITY);
240
241     memcpy(rec_position[0], state.position, sizeof(hduVector3Dd));
242     memcpy(rec_gimbalAngles[0], state.gimbalAngles, sizeof(hduVector3Dd));
243     rec_timeidx[0] = now;
244     time_next = now;
245
246     //for(t=1;t< lim_rec_time;++t)
247     t = 1;
248     do
249     {
250         time_next += INTV_REC;
251         while (now < time_next) { //wait until next time
252             now = timeGetTime();
253         }
254         hdScheduleSynchronous(GetDeviceStateCallback, &state,
255             HD_DEFAULT_SCHEDULER_PRIORITY);
256
257         memcpy(rec_position[t], state.position, sizeof(hduVector3Dd));
258         memcpy(rec_gimbalAngles[t], state.gimbalAngles, sizeof(hduVector3Dd));
259         rec_timeidx[t] = now;
260         ++t;
261     } while (!kbhit() && t < LEN_REC); // exit when key press or time limit.
262
263     end_idx_record = t;
264
265     //#if defined(WIN32)
266     //         Sleep(INTV_REC);
267     //#elif defined(linux)

```

```

268 //          struct timespec timeOut;
269 //          timeOut.tv_sec = 0;
270 //          timeOut.tv_nsec = 1 * 100000000;
271 //          nanosleep(&timeOut, NULL);
272 //endif
273
274
275 fprintf(stderr, "record end.\n");
276 /* get time difference */
277 for (t = 0; t < end_idx_record; ++t) {
278     rec_timeidx[t] -= time_start;
279 }
280
281 for (t = 0; t < end_idx_record; ++t) {
282     int i;
283     printf("time:%d: ", rec_timeidx[t]);
284     printf("Current position (mm):");
285     for (i = 0; i < 3; i++)
286     {
287         printf(" %f", rec_position[t][i]);
288     }
289     //printf("\n");
290
291     printf("Current Gimbal Angles (rad):");
292     for (i = 0; i < 3; i++)
293     {
294         printf(" %f", rec_gimbalAngles[t][i]);
295     }
296     printf("\n");
297 }
298
299 // write to file
300 fp_rec_csv = fopen(FNAME_REC, "w");
301 if (fp_rec_csv==NULL) {
302     fprintf(stderr, "file open failed! : %s \n", FNAME_REC);
303     exit(1);
304 }
305 else {
306     fprintf(fp_rec_csv, "# time[ms], position x, y, z [mm], gimbal_angle [rad] \n");
307     for (t = 0; t < end_idx_record; ++t) {
308         int i;
309         fprintf(fp_rec_csv, "%d, ", rec_timeidx[t]);
310         for (i = 0; i < 3; i++)
311         {
312             fprintf(fp_rec_csv, "%f, ", rec_position[t][i]);
313         }
314         for (i = 0; i < 3; i++)
315         {
316             fprintf(fp_rec_csv, "%f", rec_gimbalAngles[t][i]);
317             if (i < 2) {
318                 fprintf(fp_rec_csv, ", ");
319             }
320         }
321     }

```

```

322         fprintf(fp_rec_csv, "\n");
323
324     }
325     fclose(fp_rec_csv);
326 }
327 fprintf(stderr, "record end.total time %d time steps.\n",end_idx_record);
328
329 }
330
331 /*****
332 Callback that replay state.
333 *****/
334 void ReplayDeviceState()
335 {
336     DWORD now,time_replay_start,time_next;
337
338     int i;
339     int i_resample;
340
341     if (!ReplayReady) { //replay data not loaded successfully
342         fprintf(stderr, "replay data not loaded successfully.\n");
343         fprintf(stderr, "Please confirm replay data loaded by L command.\n");
344         return;
345     }
346
347     // guide to initial gravity well position
348     fprintf(stderr, "Replay mode :\n");
349     fprintf(stderr, "    Please remove stylus from calibration well, and hold any
350 position.\n");
351     fprintf(stderr, "    When you got ready, press any key.");
352     fflush(stdin); getch();
353     fprintf(stderr, "Now I guide you to the starting position.\n");
354
355     wellPos[0] = replay_position_resample[0][0];
356     wellPos[1] = replay_position_resample[0][1];
357     wellPos[2] = replay_position_resample[0][2];
358     ReplayMode = true;
359
360     fprintf(stderr, "When your stylus moved to start position, press any key.\n");
361     fflush(stdin); getch();
362
363     printf("Replay starts after 3 sec.count.");
364     for (i = 3; i > 0; --i) {
365         printf("%d ", i);
366         Sleep(1000);
367     }
368     printf("...go!\n");
369
370     // replay
371     now = time_replay_start = timeGetTime();
372
373     for (i_resample = 1; i_resample < (end_idx_resample-1); ++i_resample) {
374         time_next = time_replay_start + replay_timeidx_resample[i_resample];

```

```
375     while (now < time_next) { //wait until next time
376         now = timeGetTime();
377     }
378     // move gravity well
379     memcpy(wellPos, replay_position_resample[i_resample], sizeof(hduVector3Dd));
380
381
382 }
383
384
385
386
387 fprintf(stderr, "press any key to quit replay mode.\n");
388 fflush(stdin); getch();
389 ReplayMode = false;
390 fprintf(stderr, "replay mode disabled.\n");
391 return;
392 }
393
394
395 void read_resampled_ReplayData() {
396     FILE* fp_replay_csv = fopen(FNAME_REPLAY_RESAMPLE, "rt");
397     if (!fp_replay_csv) {
398         fprintf(stderr, "Cannot open file:%s\n", FNAME_REPLAY_RESAMPLE);
399         exit(1);
400     }
401     char linebuf[LEN_LINE_CSV];
402     int i = 0;
403     int tmpTimeMsec;
404     double tmpT, tmpX, tmpY, tmpZ;
405     // read 1-line until EOF
406     while (fgets(linebuf, LEN_LINE_CSV, fp_replay_csv) != NULL) { // until encount EOF
407
408         if (linebuf[0] != '#') { // skip header line
409             sscanf(linebuf, "%lf,%lf,%lf,%lf", &tmpT, &tmpX, &tmpY, &tmpZ);
410             // convert time to msec
411             tmpTimeMsec = tmpT;
412             replay_timeidx_resample[i] = tmpTimeMsec;
413             replay_position_resample[i][0] = tmpX;
414             replay_position_resample[i][1] = tmpY;
415             replay_position_resample[i][2] = tmpZ;
416
417             ++i;
418             if (i < MAX_LEN_REPLAY_RAW) {
419                 continue;
420             }
421             else {
422                 fprintf(stderr, "Maximum data length reached. reading file terminated.
423 \n");
424                 break;
425             }
426         }
427     }
428     // store end of index for resample
```

```

428     end_idx_resample = i + 1;
429     fprintf(stderr, "Replay resampled data loaded. total %d elements.\n",
end_idx_resample);
430     ReplayReady = true;
431     return;
432 }
433
434 void read_RecordData() {
435     FILE* fp_replay_csv = fopen(FNAME_REC, "rt");
436     if (!fp_replay_csv) {
437         fprintf(stderr, "Cannot open file:%s\n", FNAME_REC);
438         exit(1);
439     }
440     char linebuf[LEN_LINE_CSV];
441     int i = 0;
442     int tmpTimeMsec;
443     double tmpT, tmpX, tmpY, tmpZ;
444     // read 1-line until EOF
445     while (fgets(linebuf, LEN_LINE_CSV, fp_replay_csv) != NULL) { // until encount EOF
446
447         if (linebuf[0] != '#') { // skip header line
448             sscanf(linebuf, "%lf,%lf,%lf,%lf", &tmpT, &tmpX, &tmpY, &tmpZ);
449             // convert time to msec
450             tmpTimeMsec = tmpT;
451             replay_timeidx_resample[i] = tmpTimeMsec;
452             replay_position_resample[i][0] = tmpX;
453             replay_position_resample[i][1] = tmpY;
454             replay_position_resample[i][2] = tmpZ;
455
456             ++i;
457             if (i < MAX_LEN_REPLAY_RAW) {
458                 continue;
459             }
460             else {
461                 fprintf(stderr, "Maximum data length reached. reading file terminated.
\n");
462                 break;
463             }
464         }
465     }
466     // store end of index for resample
467     end_idx_resample = i + 1;
468     fprintf(stderr, "Replay resampled data loaded. total %d elements.\n",
end_idx_resample);
469
470     ReplayReady = true;
471     return;
472 }
473
474
475 void read_raw_ReplayData() {
476     FILE* fp_replay_csv = fopen(FNAME_REPLAY, "rt");
477     if (!fp_replay_csv) {
478         fprintf(stderr, "Cannot open file:%s\n", FNAME_REPLAY);

```

```

479         exit(1);
480     }
481     char linebuf[LEN_LINE_CSV];
482     int i = 0;
483     int tmpTimeMsec;
484     double tmpT,tmpX, tmpY, tmpZ;
485     // read 1-line until EOF
486     while (fgets(linebuf, LEN_LINE_CSV, fp_replay_csv) != NULL) { // until encount EOF
487
488         if (linebuf[0] != '#') { // skip header line
489             sscanf(linebuf, "%lf,%lf,%lf,%lf", &tmpT, &tmpX, &tmpY, &tmpZ);
490             // convert time to msec
491             tmpTimeMsec = tmpT * 1000;
492             replay_timeidx_raw[i] = tmpTimeMsec;
493             replay_position_raw[i][0] = tmpX;
494             replay_position_raw[i][1] = tmpY;
495             replay_position_raw[i][2] = tmpZ;
496
497             ++i;
498             if (i < MAX_LEN_REPLAY_RAW) {
499                 continue;
500             }
501             else {
502                 fprintf(stderr, "Maximum data length reached. reading file terminated.
503 \n");
504                 break;
505             }
506         }
507         fprintf(stderr, "Replay raw data loaded. total %d elements.\n", i);
508         ReplayReady = true;
509         return;
510     }
511
512
513 void resampleReplayData() {
514     int i = 0;
515     int i_resample = 0;
516     int current_time; //ms
517     int offset_time; //ms
518
519     if (!ReplayReady) { //replay data not loaded successfully
520         fprintf(stderr, "replay data not loaded successfully.abort resampling.\n");
521         return;
522     }
523
524     // set first data as timeidx 0
525     offset_time = replay_timeidx_raw[0];
526     current_time = offset_time;
527
528     replay_timeidx_resample[0] = 0;
529     memcpy(replay_position_resample[0], replay_position_raw[0], sizeof(hduVector3Dd));
530
531

```

```

532
533     for (i_resample = 1; i_resample < MAX_LEN_REPLAY_RESAMPLE; ++i_resample) {
534         // set kijun time
535         current_time += INTV_RESAMPLE_REPLAY_MS;
536         //search next time idx which exceed kijun time
537         while (replay_timeidx_raw[i] < current_time) {
538             ++i;
539             if (i >= MAX_LEN_REPLAY_RAW) {
540                 fprintf(stderr, "maximum raw replay data reached.\n");
541                 return;
542             }
543         }
544         // set resampled array
545         replay_timeidx_resample[i_resample] = replay_timeidx_raw[i];
546         memcpy(replay_position_resample[i_resample], replay_position_raw[i],
547             sizeof(hduVector3Dd));
548         fprintf(stderr, "resample :index:%d resampled time %d , raw time %d.\n",
549             i_resample, replay_timeidx_resample[i_resample], replay_timeidx_raw[i]);
550
551         // store end of index for resample
552         end_idx_resample = i_resample + 1;
553
554     }
555     return;
556 }
557
558 void writeResampledReplayData() {
559     int t;
560     FILE* fp_replay_resample;
561     // write to file
562     fp_replay_resample = fopen(FNAME_REPLAY_RESAMPLE, "w");
563     if (fp_replay_resample == NULL) {
564         fprintf(stderr, "file open failed! : %s \n", FNAME_REPLAY_RESAMPLE);
565         exit(1);
566     }
567     else {
568         fprintf(fp_replay_resample, "# time[ms], position x, y, z [mm], gimbal_angle [rad]
569 \n");
570         for (t = 0; t < end_idx_resample; ++t) {
571             int i;
572             fprintf(fp_replay_resample, "%d, ", replay_timeidx_resample[t]);
573             for (i = 0; i < 3; i++)
574             {
575                 fprintf(fp_replay_resample, "%f", replay_position_resample[t][i]);
576                 if (i < 2) {
577                     fprintf(fp_replay_resample, ", ");
578                 }
579             }
580             // for (i = 0; i < 3; i++)
581             // {
582             //     fprintf(fp_rec_csv, "%f", rec_gimbalAngles[t][i]);
583             //     if (i < 2) {
584                 fprintf(fp_rec_csv, ", ");

```

```

584 //      }
585
586     }
587     fprintf(fp_replay_resample, "\n");
588
589     }
590     fclose(fp_replay_resample);
591 }
592 }
593
594
595
596
597
598
599 /*****
600 Main function.
601 Initializes the device, starts the schedule, creates a schedule callback
602 to handle gravity well forces, waits for the user to press a button, exits
603 the application.
604 *****/
605 int main(int argc, char* argv[])
606 {
607     HDErrorInfo error;
608     /* Initialize the device, must be done before attempting to call any hd
609        functions. Passing in HD_DEFAULT_DEVICE causes the default device to be
610        initialized. */
611     HHD hHD = hdInitDevice(HD_DEFAULT_DEVICE);
612     if (HD_DEVICE_ERROR(error = hdGetError()))
613     {
614         hduPrintError(stderr, &error, "Failed to initialize haptic device");
615         fprintf(stderr, "\nPress any key to quit.\n");
616         getch();
617         return -1;
618     }
619
620     printf("Guide and Record motion Demo!\n");
621     printf("Found device model: %s.\n\n", hdGetString(HD_DEVICE_MODEL_TYPE));
622
623     if (!initDemo())
624     {
625         printf("Demo Initialization failed\n");
626         printf("Press any key to exit\n");
627         getch();
628     }
629
630
631     /* Schedule the main callback that will render forces to the device. */
632     hGravityWell = hdScheduleAsynchronous(
633         guideAndRecordCallback, 0,
634         HD_MAX_SCHEDULER_PRIORITY);
635
636     hdEnable(HD_FORCE_OUTPUT);
637     hdStartScheduler();

```

```

638
639 /* Check for errors and abort if so. */
640 if (HD_DEVICE_ERROR(error = hdGetError()))
641 {
642     hduPrintError(stderr, &error, "Failed to start scheduler");
643     fprintf(stderr, "\nPress any key to quit.\n");
644     return -1;
645 }
646
647 PrintHelp();
648
649 /* Start the main application loop */
650 mainLoop();
651
652 /* For cleanup, unschedule callback and stop the scheduler. */
653 hdStopScheduler();
654 hdUnschedule(hGravityWell);
655
656 /* Disable the device. */
657 hdDisableDevice(hHD);
658
659 return 0;
660 }
661 /*****
662 The main loop of execution. Detects and interprets keypresses. Monitors and
663 initiates error recovery if necessary.
664 *****/
665 void mainLoop()
666 {
667     int keypress;
668
669     while (TRUE)
670     {
671         if (_kbhit())
672         {
673             keypress = getch();
674             keypress = toupper(keypress);
675
676             switch (keypress)
677             {
678                 case 'L': read_resampled_ReplayData(); PrintHelp(); break;
679                 case 'G': ReplayDeviceState(); PrintHelp(); break;
680                 case 'R': RecordDeviceState(); PrintHelp(); break;
681                 case 'D': read_RecordData(); PrintHelp(); break;
682
683                 case 'P': PrintDeviceState(FALSE); PrintHelp(); break;
684                 case 'C': PrintDeviceState(TRUE); PrintHelp(); break;
685                 case 'A': read_raw_ReplayData(); resampleReplayData(); PrintHelp(); break;
686                 case 'W': writeResampledReplayData(); PrintHelp(); break;
687                 case 'H': PrintHelp(); break;
688                 case 'Q': return;
689                 default: PrintHelp(); break;
690             }
691         }

```

```

692
693     /* Check if the scheduled callback has stopped running */
694     if (!hdWaitForCompletion(hGravityWell, HD_WAIT_CHECK_STATUS))
695     {
696         fprintf(stderr, "\nThe main scheduler callback has exited\n");
697         fprintf(stderr, "\nPress any key to quit.\n");
698         getch();
699         return;
700     }
701 }
702 }
703
704 /*****
705 Servo callback.
706 Called every servo loop tick. Simulates a gravity well, which sucks the device
707 towards its center whenever the device is within a certain range.
708 *****/
709 HDCallbackCode HDCALLBACK guideAndRecordCallback(void *data)
710 {
711     const HDdouble kStiffness = 0.15; /* N/mm */ //magnitude of gravity. default is 0.075
712     const HDdouble kStylusTorqueConstant = 500; /* torque spring constant (mN.m/radian)*/
713     const HDdouble kJointTorqueConstant = 1000; /* torque spring constant (mN.m/radian)*/
714
715     const HDdouble kForceInfluence = 100; /* mm */
716     const HDdouble kTorqueInfluence = 3.14; /* radians */
717
718     /* This is the position of the gravity well in cartesian
719        (i.e. x,y,z) space. */
720     //static const hduVector3Dd wellPos(0,0,0);
721     static const hduVector3Dd stylusVirtualFulcrum(0.0, 0.0, 0.0); // In radians
722     static const hduVector3Dd jointVirtualFulcrum(0.0, 0.0, 0.0); // In radians
723
724     HDErrorInfo error;
725     hduVector3Dd position;
726
727     hduVector3Dd force;
728     hduVector3Dd positionTwell;
729     hduVector3Dd gimbalAngles;
730     hduVector3Dd gimbalTorque;
731     hduVector3Dd gimbalAngleOfTwist;
732     hduVector3Dd jointAngles;
733     hduVector3Dd jointTorque;
734     hduVector3Dd jointAngleOfTwist;
735
736     HHD hHD = hdGetCurrentDevice();
737
738     /* Begin haptics frame. ( In general, all state-related haptics calls
739        should be made within a frame. ) */
740     hdBeginFrame(hHD);
741
742     /* Get the current position of the device. */
743     hdGetDoublev(HD_CURRENT_POSITION, position);
744     hdGetDoublev(HD_CURRENT_GIMBAL_ANGLES, gimbalAngles );
745     hdGetDoublev(HD_CURRENT_JOINT_ANGLES, jointAngles );

```

```

746
747     memset(force, 0, sizeof(hduVector3Dd));
748
749
750     /* > positionTwell = wellPos-position <
751        Create a vector from the device position towards the gravity
752        well's center. */
753     hduVecSubtract(positionTwell, wellPos, position);
754
755
756     /* If the device position is within some distance of the gravity well's
757        center, apply a spring force towards gravity well's center. The force
758        calculation differs from a traditional gravitational body in that the
759        closer the device is to the center, the less force the well exerts;
760        the device behaves as if a spring were connected between itself and
761        the well's center. */
762     if (hduVecMagnitude(positionTwell) < kForceInfluence)
763     {
764         /* > F = k * x <
765            F: Force in Newtons (N)
766            k: Stiffness of the well (N/mm)
767            x: Vector from the device endpoint position to the center
768            of the well. */
769         hduVecScale(force, positionTwell, kStiffness);
770     }
771
772
773     /* Send the forces & torques to the device. */
774     /*Switch back between sending forces & torques
775     to the base motors */
776     if (!ReplayMode) {
777         force[0] = force[1] = force[2] = 0;
778     }
779     hdSetDoublev(HD_CURRENT_FORCE, force);
780
781
782     // printf("%f\t%f\t%f\n\n", jointTorque[0], jointTorque[1], jointTorque[2]);
783     /* End haptics frame. */
784     hdEndFrame(hHD);
785
786     /* Check for errors and abort the callback if a scheduler error
787        is detected. */
788     if (HD_DEVICE_ERROR(error = hdGetError()))
789     {
790         hduPrintError(stderr, &error,
791                      "Error detected while rendering gravity well\n");
792
793         if (hduIsSchedulerError(&error))
794         {
795             return HD_CALLBACK_DONE;
796         }
797     }
798
799     /* Signify that the callback should continue running, i.e. that

```

```
800     it will be called again the next scheduler tick. */
801     return HD_CALLBACK_CONTINUE;
802 }
803
804 bool initDemo(void)
805 {
806     HDErrorInfo error;
807     int calibrationStyle;
808     printf("Calibration\n");
809
810     hdGetIntegerv(HD_CALIBRATION_STYLE, &calibrationStyle);
811     if (calibrationStyle & HD_CALIBRATION_AUTO || calibrationStyle & HD_CALIBRATION_INKWE-
LL)
812     {
813         printf("Please prepare for starting the demo by \n");
814         printf("placing the device at its reset position.\n\n");
815         printf("Press any key to continue...\n");
816         getch();
817         return 1;
818     }
819     if (calibrationStyle & HD_CALIBRATION_ENCODER_RESET )
820     {
821         printf("Please prepare for starting the demo by \n");
822         printf("placing the device at its reset position.\n\n");
823         printf("Press any key to continue...\n");
824
825         getch();
826
827         hdUpdateCalibration(calibrationStyle);
828         if (hdCheckCalibration() == HD_CALIBRATION_OK)
829         {
830             printf("Calibration complete.\n\n");
831             return 1;
832         }
833         if (HD_DEVICE_ERROR(error = hdGetError()))
834         {
835             hduPrintError(stderr, &error, "Reset encoders reset failed.");
836             return 0;
837         }
838     }
839 }
840 /*****
841
```